

Basics of Cryptography – Secure Communication Prototype

IHC – Practical Approach to Cyber Security (PACS)

Modular Assignment 2

Student: Nitesh Vishwakarma

1. Introduction

This assignment presents a small secure communication prototype using Python. The objective is to understand how symmetric encryption, asymmetric encryption, hashing, digital signatures, certificates and TLS contribute to secure communication.

The practical uses an example healthcare appointment confirmation as the message. The example is only demonstration data and does not contain real patient information.

2. Historical Overview and Evolution

Cryptography started with simple substitution and transposition methods used to protect written messages. Classical methods were mainly designed to hide the meaning of a message.

With computers and networked systems, stronger mathematical techniques became necessary. Symmetric-key cryptography remained important because the same secret key can efficiently protect large amounts of data. Modern examples include AES.

Public-key or asymmetric cryptography introduced a pair of related keys: a public key and a private key. It made key exchange and digital signatures practical without requiring the private key to be shared. RSA is a well-known example.

Hash functions then became important for integrity checking, while digital signatures combined hashing with asymmetric cryptography to provide evidence of origin and integrity. Modern secure communication combines these primitives with certificates and protocols such as TLS.

3. System / Communication Scenario

Sender: a client application

Receiver: a healthcare appointment application

Message: "Healthcare appointment confirmation: APPT-2026-001"

The prototype demonstrates four separate cryptographic operations:

- AES-256-GCM for symmetric encryption.
- RSA-2048 OAEP for asymmetric encryption.
- SHA-256 for integrity checking.
- RSA-PSS with SHA-256 for digital signatures.

In a real system, TLS would protect the network connection and certificates would help the client verify the identity of the server.

4. Symmetric Cryptography – AES

Symmetric cryptography uses one shared secret key for encryption and decryption. It is efficient and suitable for protecting larger amounts of data.

The prototype uses AES-GCM with a 256-bit key. GCM is an authenticated-encryption mode, so the demonstration provides confidentiality and an integrity check for the encrypted message. A fresh random nonce is generated for each encryption operation.

Workflow:

1. Generate a random AES key.
2. Generate a unique nonce.

3. Encrypt the message.
4. Decrypt using the same key and nonce.
5. Compare the recovered plaintext with the original message.

Expected result: the decrypted message is identical to the original message.

5. Asymmetric Cryptography – RSA

Asymmetric cryptography uses a public/private key pair. The public key can be shared, while the private key must remain secret.

The prototype generates a 2048-bit RSA key pair. RSA-OAEP with SHA-256 is used for the demonstration.

Workflow:

1. Generate RSA private key.
2. Derive the corresponding public key.
3. Encrypt the message using the public key.
4. Decrypt using the private key.
5. Confirm that the original message is recovered.

RSA is computationally more expensive than symmetric encryption, so real systems commonly use asymmetric cryptography for key establishment and symmetric cryptography for bulk data protection.

6. Hash Function – SHA-256

A cryptographic hash function converts input data into a fixed-length digest. SHA-256 produces a 256-bit digest, normally represented as 64 hexadecimal characters.

A hash is not encryption because the original message is not recovered from the digest during normal use. The prototype calculates a SHA-256 digest for the original message and then calculates another digest after changing the message. The two values are different, demonstrating how hashing can detect modification.

7. Digital Signature

A digital signature is generated with a private key and verified with the corresponding public key. It helps provide origin authentication, integrity protection and support for non-repudiation.

The prototype uses RSA-PSS with SHA-256. The sender signs the message with the private key. The receiver verifies the signature using the public key.

If the message is changed after signing, verification should fail. This is different from encryption: a digital signature does not provide confidentiality by itself.

8. Certificates and TLS

A digital certificate binds an identity, such as a website or server name, to a public key and is normally issued within a certificate trust chain.

TLS is the standard protocol used to create a protected connection between clients and servers. During a normal TLS connection, the server presents a certificate. The client validates the certificate and establishes cryptographic session keys. After the handshake, application data is protected using the negotiated secure session.

In a real healthcare application, TLS would protect API traffic between the client and the service. Certificate validation helps the client avoid connecting to an impostor server. TLS therefore complements, rather than replaces, application-level authentication and authorization.

9. Implementation Steps

1. Install Python and the cryptography package.
2. Save the supplied Python source file.
3. Run the program from a terminal.

4. Enter/use the demonstration message defined in the script.
5. Observe AES encryption/decryption.
6. Observe RSA encryption/decryption.
7. Observe the SHA-256 digest.
8. Observe digital-signature creation and verification.
9. Observe the integrity test after modifying the message.

Command:

```
pip install cryptography
```

```
python MA2_Cryptography_Implementation_Nitesh_Vishwakarma.py
```

10. Results and Observations

The prototype demonstrates the expected cryptographic workflows. Symmetric encryption recovers the original plaintext using the same secret key. RSA demonstrates public-key encryption and private-key decryption. SHA-256 produces a fixed-length digest and produces a different digest when the input changes. RSA-PSS signature verification succeeds for the original message.

The generated ciphertext and signatures are not fixed values because secure cryptographic operations use randomly generated keys, nonces or padding. Therefore the exact encrypted strings can change every time the program is executed while the functional results remain the same.

11. Security Considerations

- Private keys and symmetric keys must never be exposed or hard-coded in production systems.
- Cryptographic randomness must come from secure random generators.
- Strong, current algorithms and appropriate key sizes should be selected.
- TLS certificates must be validated rather than blindly trusted.
- Hashes should be used for integrity purposes, not as a replacement for encryption.
- Digital signatures provide integrity and authenticity properties but do not by themselves provide confidentiality.
- In production, cryptographic key storage, rotation, access control and backup must be handled securely.

12. Conclusion

This practical helped demonstrate the different purposes of common cryptographic primitives. AES is efficient for protecting data, RSA demonstrates public/private key operations, SHA-256 provides a compact integrity representation, and digital signatures provide authenticity and integrity verification. Certificates and TLS connect these concepts to secure real-world network communication.

The prototype is intentionally small so that each cryptographic operation can be observed separately and understood before applying the concepts to larger cloud and security systems.

13. References

1. NIST Cryptographic Standards and Guidelines.
2. NIST Cybersecurity and Privacy Reference resources.
3. Python Cryptography Library documentation.
4. Python hashlib documentation.
5. Assignment-provided Secure Data Communication Dataset (Kaggle) / public dataset resource.
6. RFC 8446 – The Transport Layer Security (TLS) Protocol Version 1.3.

Appendix A – Source Code

```
"""
IHC - Practical Approach to Cyber Security
Modular Assignment 2: Basics of Cryptography

Student: Nitesh Vishwakarma

This practical demonstrates:
1. AES symmetric encryption/decryption
2. RSA asymmetric encryption/decryption
3. SHA-256 hashing for integrity
4. RSA-PSS digital signature and verification

Install:
""" pip install cryptography

from cryptography.fernet import Fernet
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import padding, rsa
from cryptography.hazmat.primitives.asymmetric import utils
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

def symmetric_encryption_demo(message: str):
    # AES-GCM provides confidentiality and integrity for the message.
    key = AESGCM.generate_key(bit_length=256)
    aes = AESGCM(key)
    nonce = os.urandom(12)

    ciphertext = aes.encrypt(nonce, message.encode(), None)
    plaintext = aes.decrypt(nonce, ciphertext, None).decode()

    return key, nonce, ciphertext, plaintext

def rsa_demo(message: str):
    # RSA is used here to demonstrate asymmetric encryption.
    private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
    public_key = private_key.public_key()

    ciphertext = public_key.encrypt(
        message.encode(),
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None,
        ),
    )

    plaintext = private_key.decrypt(
        ciphertext,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None,
        ),
    ).decode()

    return private_key, public_key, ciphertext, plaintext

def sha256_demo(message: str):
    digest = hashlib.sha256(message.encode()).hexdigest()
    return digest

def signature_demo(message: str):
    private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
    public_key = private_key.public_key()

    signature = private_key.sign(
        message.encode(),
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH,
        ),
        hashes.SHA256(),
    )

    public_key.verify(
        signature,
        message.encode(),
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH,
        ),
        hashes.SHA256(),
    )

    return signature, "Signature verified successfully."

if __name__ == "__main__":
    import os

    message = "Healthcare appointment confirmation: APPT-2026-001"

    print("=" * 60)
    print("BASICS OF CRYPTOGRAPHY - PRACTICAL DEMONSTRATION")
    print("=" * 60)
```

```

print("Input message:", message)

# 1. Symmetric encryption
key, nonce, ciphertext, decrypted = symmetric_encryption_demo(message)
print("\n[1] AES-256-GCM")
print("Encrypted data:", base64.b64encode(ciphertext).decode())
print("Decrypted data:", decrypted)
print("Result: Encryption/decryption successful")

# 2. Asymmetric encryption
private_key, public_key, rsa_ciphertext, rsa_plaintext = rsa_demo(message)
print("\n[2] RSA-2048 OAEP")
print("Encrypted data:", base64.b64encode(rsa_ciphertext).decode())
print("Decrypted data:", rsa_plaintext)
print("Result: Encryption/decryption successful")

# 3. Hash
digest = sha256_demo(message)
print("\n[3] SHA-256")
print("Message hash:", digest)
print("Result: Hash generated successfully")

# 4. Digital signature
signature, result = signature_demo(message)
print("\n[4] RSA-PSS Digital Signature")
print("Signature:", base64.b64encode(signature).decode())
print("Verification:", result)

# Integrity check example
modified_message = message + " - modified"
modified_digest = sha256_demo(modified_message)
print("\n[5] Integrity check")
print("Original hash:", digest)
print("Modified hash:", modified_digest)
print("Hashes match:", digest == modified_digest)
print("Observation: changing the message changes the hash.")

```